

Buscando al Coronel Kurtz

Fundamentos de la Inteligencia Artificial – IMAT 25/26

Ignacio Mascarell Agúndez

202409248@alu.comillas.edu

Parte 1 – Búsqueda No Informada

Introducción

Este proyecto consiste en programar un agente, llamado Capitán Willard, que debe moverse por un palacio, encontrar al coronel Kurtz y salir con vida, tomando decisiones solo con la información parcial que percibe en cada paso. El mapa se genera de forma aleatoria y el agente no conoce de entrada dónde están los peligros ni la salida, así que tiene que ir deduciendo las casillas seguras a medida que explora.

A lo largo de esta sección de la memoria se describe los siguientes apartados: el entorno del problema, el almacenamiento del conocimiento, el razonamiento basado en los perceptos y la toma de decisiones siempre en modo manual.

Descripción del entorno

El palacio se modela como un tablero de tamaño 6×6 y el agente empieza siempre en la esquina superior izquierda, que en índices de Python corresponde a (0,0). En el tablero se colocan aleatoriamente tres precipicios (trampas de caída), un soldado enemigo, la posición del coronel Kurtz y una casilla de salida, sin solaparse entre sí, es decir, estas posiciones no pueden coincidir.

Durante el desarrollo del juego, si el agente entra en una casilla de peligro muere. Dichas casillas de peligro son las de los precipicios y el soldado enemigo (en caso de estar vivo). Por otro lado, el soldado dispone de una granada única. Esta granada solo se puede lanzar desde una casilla adyacente. Si el agente acierta el disparo y lanza la granada a la casilla del soldado enemigo, lo elimina.

Representación del conocimiento

El conocimiento del agente es un aspecto clave en el desarrollo del código. Con el conocimiento deducido y almacenado y la inferencia lógica, el agente es capaz de informar al usuario para que este lleva a cabo la toma de decisiones. Lógicamente, el usuario buscará elegir las acciones más beneficiosas o menos perjudiciales, dependiendo de la situación, y así resolver el problema.

El agente mantiene un estado simple. Almacena información sobre su posición actual, si aún conserva la granada y si ya ha logrado encontrar al coronel Kurtz. De esta manera el usuario puede elegir su siguiente objetivo.

El almacenamiento del conocimiento del agente se hace de la siguiente manera. En primer lugar, se guardan diferentes conjuntos de casillas, que representan distintos tipos (seguras o visitadas, con precipicios, con soldado y sospechosas). De esta manera el agente es capaz de distinguir entre las casillas a las que puede acceder sin correr ningún tipo de riesgo mediante inferencia lógica. Por otra parte, también se mantiene información sobre la casilla de salida. Se almacena un conjunto de candidatos a ser la casilla de salida que, a medida que avanza el juego, va perdiendo tamaño.

Inferencia lógica

La inferencia lógica es la parte fundamental en la toma de decisiones del agente. Junto con el conocimiento almacenado llevan a cabo el filtrado de información para que el usuario sea capaz de tomar la decisión adecuada en cada situación.

En cada iteración el entorno devuelve una lista de 8 booleanos con los perceptos:

[Brisa (Precipicio), Ronquido (Soldado Enemigo), Resplandor (Salida), Pared UP, Pared DOWN, Pared LEFT, Pared RIGHT, Grito (Soldado Enemigo Muerto)]

Con los perceptos obtenidos el agente razona sobre las características de las casillas adyacentes. Si no hay brisa o no hay ronquido, el agente marca como seguras (para ese peligro) las casillas adyacentes; si sí hay brisa/ronquido, guarda como “posible peligro” las adyacentes que todavía no puede descartar, y cuando por acumulación de información solo queda un candidato, lo deduce como peligro confirmado.

En adición, también se encuentran perceptos sobre lo que va ocurriendo durante la partida. Cuando se oye el grito, el agente pasa a considerar que el soldado ya no es una amenaza y actualiza la variable ‘soldier_dead’ y marca el resto de posiciones como sin soldado. Por otra parte, el resplandor se usa para intersecar o eliminar candidatos de salida según esté activo o no en la posición actual y sus vecinas.

Variantes de juego

El programa ofrece una única variante de juego: el modo manual. En este modo, en cada iteración se le presentan los perceptos al usuario a través de la terminal y este mismo el que decide la acción a ejecutar. Además, se representa el tablero, sus casillas y sus respectivas clasificaciones.

Las acciones posibles incluyen moverse (W/A/S/D), lanzar granada en una dirección (GW/GA/GS/GD) y la acción de salir (X), que solo completa la misión si se ejecuta estando en la casilla de salida y habiendo encontrado antes a Kurtz. En adición, el entorno y la partida se ejecutan en un bucle con un límite de pasos para evitar ejecuciones infinitas en situaciones donde no queden acciones seguras claras.

Algoritmo de decisión

Como se ha explicado anteriormente, la toma de decisiones la lleva a cabo el usuario. Para ello, se sigue el siguiente proceso. En primer lugar, se obtienen los perceptos de la posición. Posteriormente se actualiza el conocimiento del agente y se presentan las deducciones realizadas a través de la inferencia lógica. Finalmente, el usuario selecciona la acción a ejecutar por la terminal.

La información obtenida a través de la inferencia se presenta en cada iteración en la terminal. Por una parte, se pinta el tablero, representando la clasificación deducida de las casillas vecinas a la de ese preciso momento del agente. Por otro lado, se presenta información relacionada con la casilla de salida deducida, el estado del soldado enemigo (vivo/muerto) y si se ha reencontrado con el coronel Kurtz.

Resultados

A continuación, se presentan unas capturas de pantallas de los resultados obtenidos al ejecutar el código de 'kurtz.py'. Se puede observar el funcionamiento del código: la localización del coronel Kurtz, el uso de la granada, la salida del palacio, todo ello en modo manual.

```

----- STEP 2 -----
Current position: (0, 1)

--- The board ---
O W O . . .
. O . . . .
. . . . .
. . . . .
. . . . .
. . . . .

- Percepts: WALL UP

--- Current information ---
- Grenade: YES
- Found Murtz: NO
- Possible exits: 31

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: X

Select action: |

```

Ejecución básica

```

----- STEP 6 -----
Current position: (1, 4)

--- The board ---
O O . ? .
O O O W ?
. . . ? .
. . . . .
. . . . .
. . . . .

- Percepts: SMOKE

--- Current information ---
- Grenade: YES
- Found Murtz: NO
- Possible exits: 21

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: X

Select action: |

```

```

----- STEP 13 -----
Current position: (1, 5)

--- The board ---
O O . O O O
O O O W O
. . . O S P
. . . . .
. . . . .
. . . . .

- Percepts: BREEZE | WALL RIGHT

--- Current information ---
- Grenade: YES
- Found Murtz: YES
- Possible exits: 18

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: X

Select action: |

```

Deducción y clasificación de casillas

```

----- STEP 14 -----
Current position: (1, 4)

--- The board ---
O O . O O O
O O O W O
. . . O S P
. . . . .
. . . . .
. . . . .

- Percepts: SMOKE

--- Current information ---
- Grenade: YES
- Found Murtz: YES
- Possible exits: 18

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: X

Select action: GS

```

```
----- STEP 15 -----
Current position: (1, 4)

--- The board ---
0 0 . 0 0 0
. 0 0 0 W 0
. . . 0 0 P
. . . . .
. . . . .
. . . . .
. . . . .

- Percepts: SCREAM

--- Current information ---

- Grenade: NO
- Found Murtz: YES
- Possible exits: 18

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GM | GA | GS | GD
- Exit: X

Select action: |
```

Eliminación de soldado enemigo con granada

```
----- STEP 46 -----
Current position: (4, 4)

--- The board ---
0 0 . 0 0 0
0 0 0 0 0 0
0 0 0 0 0 P
0 0 0 0 0 .
0 0 0 0 0 W
0 0 0 0 0 .
0 0 0 0 0 .

- Percepts: None obtained

--- Current information ---

- Grenade: NO
- Found Murtz: YES
- Confirmed exit in: (5, 1)

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GM | GA | GS | GD
- Exit: X

Select action:
```

Deducción de salida

```
----- STEP 50 -----
Current position: (5, 1)

--- The board ---
0 0 . 0 0 0
0 0 0 0 0 0
0 0 0 0 0 P
0 0 0 0 . .
0 0 0 0 . .
0 0 0 0 . .
0 W 0 0 . .

- Percepts: GLOW | WALL DOWN

--- Current information ---

- Grenade: NO
- Found Murtz: YES
- Confirmed exit in: (5, 1)

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GM | GA | GS | GD
- Exit: X

Select action: X
Result: WIN
```

Salida y victoria

```
----- STEP 43 -----
Current position: (5, 1)

--- The board ---
0 . . . .
0 . . . .
0 0 0 . .
0 0 0 0 .
0 P 0 0 .
? W 0 0 .

- Percepts: BREEZE | WALL DOWN

--- Current information ---
- Grenade: YES
- Found Murtz: NO
- Possible exits: 12

----- All possible action -----
- Movement: M | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: X

Select action: A
Result: DEAD, FALL
```

Agente sólido

Uso de la IA

Si bien es cierto que se ha usado inteligencia artificial a lo largo de este proyecto, se puede asegurar que el trabajo entregado es completamente personal. En el siguiente párrafo se explica la función que ha tenido la inteligencia artificial en esta parte.

En primer lugar, la IA ha tenido un papel fundamental para entender el enunciado y verificar que el código realizado cumple por completo con las instrucciones dadas en el enunciado. Esto se debe a que uno de mis grandes fallos en la entrega anterior fue no entender correctamente el enunciado, provocando que entregase códigos que no se adaptaban a las condiciones especificadas. Por otro lado, la IA también me ha ayudado con el código. Me ha detectado errores y, mayoritariamente, puntos de mejora en temas de estética y de eficiencia, tanto de código puramente como de la estructura de este.

Parte 2 – Razonamiento probabilístico y Markov

Introducción

En la memoria de la segunda parte de este proyecto se presenta el desarrollo y funcionamiento del código que he usado para resolver el problema planteado por el enunciado. También se describe el proceso llevado a cabo para modelar el problema y la estrategia de resolución.

Esta segunda parte del proyecto se divide en dos secciones: agente bayesiano y procesos de decisión de Markov. Para la primera sección, se amplía el escenario original del palacio lógico de la primera parte del proyecto para incorporar incertidumbre explícita mediante inferencia bayesiana. Se desarrolla un agente bayesiano que navega por un tablero 6x6 con trampas mortales. El objetivo se mantiene respecto a la primera parte del proyecto: encontrar al coronel kurtz y salir con vida del palacio. Esta sección se lleva a cabo en un archivo llamado 'palace.py'.

Por otro lado, la segunda sección de esta parte se trata del desarrollo de una toma de decisiones secuencial modelada como un Proceso de Decisión de Markov (MDP) en el archivo 'river_mdp.py'. En esta sección el agente debe tratar de llegar a la salida ubicada en el otro lado del tablero. Para ello deberá superar los obstáculos producidos por las corrientes del río.

Objetivos

El objetivo principal de esta parte del proyecto es diseñar y analizar agentes capaces de actuar de forma racional bajo incertidumbre, combinando modelos probabilísticos y técnicas de planificación óptima.

En el palacio se logra que el Capitán Willard mantenga y actualice mapas de probabilidad sobre la posición de cada tipo de trampa, del soldado enemigo y de la salida, tomando decisiones en función de un umbral de riesgo configurable en lugar de exigir seguridad absoluta. En el río se busca modelar el problema como un MDP completamente especificado (estados, acciones, transiciones y recompensas), aplicar Value Iteration para aproximar los valores de los estados y derivar una política que minimice simultáneamente el riesgo de caer en islas peligrosas y el número esperado de pasos hasta la orilla de salida.

Diseño general

El proyecto se divide en 2 scripts principales: 'palacio.py' y 'river_mdp.py'. El primer script permite simular el problema del rescate del coronel Kurtz descrito anteriormente. Por otro lado, el segundo script simula y resuelve el problema del río modelado como un proceso de decisión de Markov.

En ambos scripts se representan los problemas como una matriz cuadrada de 6x6 y se implementa un bucle iterativo para resolverlos. En cada iteración de dicho bucle se obtiene información exterior y, según esta, se selecciona una acción a ejecutar. En el caso del palacio se aplica inferencia bayesiana. En cambio, en el caso del río, se aplica value iteration para obtener la política óptima.

Palacio: agente bayesiano

El desarrollo del agente bayesiano se lleva a cabo en el archivo 'palace.py'. Se comienza elaborando el entorno como un tablero cuadrado de tamaño 6x6 donde se colocan tres tipos de trampas (fuego, pinchos y dardos), un soldado enemigo, la salida y la posición del coronel Kurtz. Todas las posiciones son generadas aleatoriamente y se confirma que sean válidas. Por un lado, las 3 trampas pueden coincidir en la misma celda, mientras que el soldado, la salida y Kurtz se eligen en celdas inicialmente seguras. En adición, estos últimos elementos también puedan compartir posición.

Durante la ejecución, el agente recibe en cada paso un vector de perceptos booleanos. Este vector representa los estímulos que recibidos. Entre los estímulos encontramos: los asociados a cada tipo de trampa, al soldado, a la salida, la información de paredes alrededor y un indicador de grito cuando la granada ha eliminado al enemigo. Con los perceptos el agente es proporcionado toda la evidencia necesaria para actualizar sus creencias y tomar una decisión.

Antes de la toma de decisiones, el agente BayesianWillard actualiza sus creencias. Concretamente, las creencias relacionadas con las trampas, el soldado enemigo y la salida, se almacenan en una matriz de probabilidades. Se mantienen 3 matrices de probabilidad separadas para cada trampa (F, P, D), para el soldado (ES) y para la salida (ET). Dichas matrices se inicializan con un prior uniforme sobre todas las celdas excepto la de inicio. Durante la ejecución, a medida que el agente recibe nuevos perceptos, aplica inferencia bayesiana y actualiza las matrices de probabilidades. Este proceso se realiza a través de la función 'bayes_update' y también toca otros aspectos como la normalización de las distribuciones de probabilidad y el descarte de peligro en posiciones. En adición, con las 3 matrices de probabilidad mencionadas se construye un mapa de riesgo total al que tanto el modo manual como automático tienen acceso.

Para la toma de decisiones el agente lleva a cabo la siguiente política. Se combina un umbral de riesgo adaptativo con heurísticas simples sobre vecinos visitados. En cada paso, el agente analiza las celdas adyacentes y prioriza moverse a posiciones no visitadas con riesgo estimado por debajo de un umbral p . El umbral p comienza en un valor conservador y aumenta gradualmente cuando no existen opciones seguras, permitiendo decisiones en situaciones complicadas asumiendo riesgo controlado. Si no hay nuevos vecinos aceptables, el agente retrocede hacia celdas previamente visitadas de menor riesgo, y solo cuando no hay alternativas totalmente seguras selecciona de forma explícita la opción menos peligrosa aplicando una penalización a las casillas ya exploradas para evitar bucles.

El diseño incluye además una gestión explícita de la granada: cuando el percepto del soldado es verdadero y la granada está disponible, el agente estima en qué vecino es más probable encontrar al enemigo y dispara en esa dirección. En caso de eliminar al soldado enemigo, su correspondiente mapa de probabilidad se actualiza a una matriz nula. Finalmente, la salida se

intenta únicamente cuando Kurtz ha sido encontrado y la probabilidad de que la celda actual sea efectivamente la salida supera un umbral alto, evitando errores. Dichos errores son registrados para no volver a cometerlos. Durante toda la ejecución se generan automáticamente mapas de calor para las probabilidades de trampas, soldado y salida, permitiendo visualizar la evolución del razonamiento bayesiano a medida que el capitán explora el palacio.

Río: Procesos de Markov

En la segunda sección de esta parte del proyecto se desarrolla el problema del cruce del río. El agente comienza en una casilla segura y debe atravesar el tablero hasta llegar a la casilla de salida. En este caso el único tipo de acción que puede realizar es de movimiento. Al ser un problema de Markov, dichas acciones no son deterministas, sino que van marcadas por una probabilidad. Esta probabilidad viene definida por las corrientes del río generadas aleatoriamente dado un intervalo.

El tablero se modela como una matriz 7x6. La posición inicial del agente es siempre la esquina superior izquierda. El agente debe buscar la salida, que se encuentra en una posición aleatoria de la columna de la derecha. En adición, existen 2 posiciones distintas de peligro que se generan aleatoriamente. Estas posiciones de peligro son representadas como una isla y funcionan como un obstáculo, es decir, el agente no puede acceder a ellas. Por lo tanto, debe bordearlas.

Como se ha mencionado anteriormente, las acciones no son deterministas. Estas acciones son de movimiento: UP, DOWN, LEFT, RIGHT y STAY. La probabilidad de realizar con éxito la acción elegida se basa en sus probabilidades, que vienen definidas por la corriente de cada columna. La corriente de cada columna se establece de manera aleatoria y se almacena en una lista. En adición, debido a la corriente de cada columna, en caso de que la acción no sea DOWN, la corriente puede desplazar hacia abajo al agente. En cambio, si la acción es DOWN, entonces está se cumplirá con certeza ya que la corriente es la misma a lo largo de toda la columna.

Para resolver correctamente el problema el agente necesita obtener la política óptima. La política óptima del MDP se obtiene a través del algoritmo de value iteration. Se itera entre las posiciones de la matriz y en cada estado se calcula su correspondiente valor maximizando las acciones. Este valor representa el máximo valor esperado entre las acciones disponibles. Para obtener dicho valor máximo se utiliza la ecuación de Bellman.

$$V(s) = \max_a \sum_{s'} P(s'|s, a)(R(s') + \gamma * V(s'))$$

La probabilidad $P(s'|s, a)$ viene dada por la corriente de cada casilla, marcada por su columna. Una vez se ha iterado en todas las posiciones de la matriz y se ha obtenido el valor de cada nodo. Este proceso se repite hasta que el cambio máximo de los valores sea menor que el umbral o, casi igual a 0. Con el value iteration completado se calcula la política óptima eligiendo en cada estado la acción que maximiza el valor esperado.

Finalmente, una vez se ha obtenido la política óptima, el código simula una ejecución siguiendo esta misma. Se mantiene un conteo global de la recompensa recibida en cada paso y se presenta el resultado final una vez la ejecución ha terminado.

Resultados: agente bayesiano

A continuación, se presentan los resultados de la ejecución de ‘palace.py’. Se adjuntan imágenes del desarrollo del código y los resultados tras 20 ejecuciones.

Ejemplo de desarrollo de código en modo manual:

```
(base) C:\Users\imasc\OneDrive - Universidad Pontificia Comillas\imat\segundo curso\primer cuatri\fia\pr_final\p2>python palace.py
----- BAYESIAN PALACE -----
Games mode:
  1. Manual mode.
  2. Automatic mode.
Select a game mode (1/2): 1

----- STEP 1 -----
W   0.00 0.12 0.12 0.12 0.12 0.12
0.00 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12

- Percepts: WALL UP | WALL LEFT

--- Current information ---
- Grenade: YES
- Found Kurtz: NO

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E

Action: |
```

Ejecución básica

```
----- STEP 3 -----
0.00 0.00 0.00 0.14 0.14 0.14
0.00 W   0.00 0.14 0.14 0.14
0.14 0.00 0.14 0.14 0.14 0.14
0.14 0.14 0.14 0.14 0.14 0.14
0.14 0.14 0.14 0.14 0.14 0.14
0.14 0.14 0.14 0.14 0.14 0.14
0.14 0.14 0.14 0.14 0.14 0.14

- Percepts: None obtained

--- Current information ---
- Grenade: YES
- Found Kurtz: NO

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E

Action: |
```

Actualización de probabilidades

```

----- STEP 11 -----
0.00 0.00 0.00 1.00 0.18 0.18
0.00 0.00 0.00 W 1.00 0.18
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.18 0.00 0.00 0.00 0.00 0.18
0.18 0.18 0.18 0.18 0.18 0.18

- Percepts: DARTS | ENEMY SOLDIER

--- Current information ---

- Grenade: YES
- Found Kurtz: NO

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E

Action:

```

Deducciones realizadas

```

----- STEP 36 -----
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 1.00 0.00 0.00 0.00 W
0.00 0.00 0.00 1.00 0.00 1.00

- Percepts: ENEMY SOLDIER | WALL RIGHT

--- Current information ---

- Grenade: YES
- Found Kurtz: NO

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E

Action: GS
Screen: The enemy soldier has been eliminated.

----- STEP 37 -----
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00
0.00 1.00 0.00 0.00 0.00 W
0.00 0.00 0.00 1.00 0.00 0.00

- Percepts: WALL RIGHT | SCREAM

--- Current information ---

- Grenade: NO
- Found Kurtz: NO

----- All possible action -----

- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E

Action:

```

Uso de granada

```

----- STEP 54 -----
0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 1.00 0.00 0.00 W 0.00
0.00 0.00 0.00 1.00 0.00 0.00

- Percepts: GLOW
--- Current information ---
- Grenade: NO
- Found Kurtz: YES

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E
Action: W

----- STEP 55 -----
0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 W 0.00
0.00 1.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 1.00 0.00 0.00 0.00

- Percepts: GLOW
--- Current information ---
- Grenade: NO
- Found Kurtz: YES

----- All possible action -----
- Movement: W | A | S | D
- Grenade: GW | GA | GS | GD
- Exit: E
Action: X
Result: WIN

```

Salida

Ejemplo de desarrollo de código en modo automático:

```

----- BAYESIAN PALACE -----
Games mode:
1. Manual mode.
2. Automatic mode.
Select a game mode (1/2): 2

----- STEP 1 -----
W 0.00 0.12 0.12 0.12 0.12 0.12
0.00 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12
0.12 0.12 0.12 0.12 0.12 0.12

- Percepts: WALL UP | WALL LEFT
--- Current information ---
- Grenade: YES
- Found Kurtz: NO
Chosen action (AUTO): D

```

Ejecución básica

```

----- STEP 4 -----
0.00 0.00 W 0.50 0.10 0.10
0.00 0.00 0.50 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10

- Percepts: ENEMY SOLDIER | WALL UP
--- Current information ---
- Grenade: NO
- Found Kurtz: NO
Chosen action (AUTO): A

----- STEP 5 -----
0.00 W 0.00 0.50 0.10 0.10
0.00 0.00 0.50 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10

- Percepts: WALL UP
--- Current information ---
- Grenade: NO
- Found Kurtz: NO
Chosen action (AUTO): S

```

Actualización de probabilidades

```

----- STEP 3 -----
0.00 0.00 0.00 0.50 0.10 0.10
0.00 0.00 0.50 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
0.10 0.10 0.10 0.10 0.10 0.10
- Percepts: ENEMY SOLDIER | WALL UP
--- Current information ---
- Grenade: YES
- Found Muirz: NO
Chosen action (AUTO): GD
The grenade exploded, nothing happened.

```

Uso de granada

```

----- STEP 17 -----
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 1.00 0.00 0.00
0.00 0.00 0.00 0.00 1.00 0.00
0.11 0.00 0.00 0.00 0.00 0.00
0.11 0.11 0.00 0.00 0.00 0.00
0.11 0.11 0.11 0.11 0.11 0.11
- Percepts: GLOW | WALL UP | WALL RIGHT
--- Current information ---
- Grenade: NO
- Found Muirz: YES
Chosen action (AUTO): A

----- STEP 18 -----
0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 1.00 1.00 0.00 0.00
0.00 0.00 0.00 0.00 1.00 0.00
0.11 0.00 0.00 0.00 0.00 0.00
0.11 0.11 0.00 0.00 0.00 0.00
0.11 0.11 0.11 0.11 0.11 0.11
- Percepts: GLOW | WALL UP
--- Current information ---
- Grenade: NO
- Found Muirz: YES
Chosen action (AUTO): X
Result: WIN

```

Salida

Resultados obtenidos tras 20 ejecuciones:

- Victorias: 95%
- Muertes: 5%
- Media de pasos por ejecución: 40'21

- Intervalo de pasos: [9, 66]
- Pasos totales en cada ejecución: [9, 18, 20, 26, 27, 32, 32, 39, 41, 43, 46, 46, 48, 49, 53, 53, 56, 60, 66]

Resultados: MDP

A continuación, se presentan los resultados de la ejecución de 'river_mdp.py'. Se adjuntan imágenes del desarrollo del código y los resultados tras 20 ejecuciones.

Ejemplo de desarrollo del código:

```
(base) C:\Users\imasc\OneDrive - Universidad Pontificia Comillas\imat\segundo curso\primer cuatri\fia\pr_final\p2>python river_mdp.py

--- RIVER MDP ---

----- THE BOARD -----
S - - - -
- - - -
- - - -
- - - -
- - - - I -
- - - I -
- - - - E

Currents: [0.0, 0.6, 0.4, 0.4, 0.4, 0.0]

----- BEST POLICY -----
S R D R R D
R R D R R D
R R D L R D
R R D L R D
R R D L I D
R R D L I D
R R D I R D
R R R R R E

- Legend -
S: start
E: exit
I: island
U: up
D: down
R: right
L: left
```

Representación del problema y política óptima obtenida

```
Simulating execution:
1. Position: (0, 0) | Action: right | Next position: (0, 1) | Reward: -1
2. Position: (0, 1) | Action: right | Next position: (0, 2) | Reward: -1
3. Position: (0, 2) | Action: down | Next position: (1, 2) | Reward: -1
4. Position: (1, 2) | Action: down | Next position: (2, 2) | Reward: -1
5. Position: (2, 2) | Action: down | Next position: (3, 2) | Reward: -1
6. Position: (3, 2) | Action: down | Next position: (4, 2) | Reward: -1
7. Position: (4, 2) | Action: down | Next position: (5, 2) | Reward: -1
8. Position: (5, 2) | Action: down | Next position: (6, 2) | Reward: -1
9. Position: (6, 2) | Action: right | Next position: (6, 2) | Reward: -1
10. Position: (6, 2) | Action: right | Next position: (6, 2) | Reward: -1
11. Position: (6, 2) | Action: right | Next position: (6, 2) | Reward: -1
12. Position: (6, 2) | Action: right | Next position: (6, 2) | Reward: -1
13. Position: (6, 2) | Action: right | Next position: (6, 3) | Reward: -1
14. Position: (6, 3) | Action: right | Next position: (6, 3) | Reward: -1
15. Position: (6, 3) | Action: right | Next position: (6, 4) | Reward: -1
16. Position: (6, 4) | Action: right | Next position: (6, 5) | Reward: 100

Success.
Steps: 16
Total reward: 85
```

Aplicación de política óptima y cálculo de recompensa

Resultados obtenidos tras 20 ejecuciones:

- Recompensa media: 76'65
- Intervalo de recompensa: [61, 86]
- Resultado de todas las ejecuciones: [83, 70, 71, 79, 72, 81, 84, 76, 86, 77, 73, 72, 61, 77, 83, 80, 85, 84, 77, 62]

Uso de la IA

Si bien es cierto que se ha usado inteligencia artificial a lo largo de este proyecto, se puede asegurar que el trabajo entregado es completamente personal. En los siguientes párrafos se explica la función que ha tenido la inteligencia artificial en esta parte.

En la primera sección del agente bayesiano la inteligencia artificial me ha ayudado en los siguientes puntos. En primer lugar, al igual que con el agente lógico, la IA ha tenido un papel fundamental ayudándome a entender el enunciado y verificando que mi trabajo cumpliera con las indicaciones dadas en el mismo. Esto se debe a que, uno de mis grandes fallos en mi primera entrega fue no comprender correctamente el enunciado, lo que me llevaba a entregar códigos que no se adaptaban a lo pedido. Por otro lado, en el código me ha ayudado a: estructurarlo, corregir errores, sugerir mejoras. De esta manera mi he logrado un código más eficiente y estético. Finalmente, en el apartado en el que, con enorme diferencia, más ayuda de la IA he necesitado, ha sido en el guardado de los heatmaps.

Por otra parte, en la sección de los procesos de decisión de Markov, la inteligencia artificial me ha ayudado en los siguientes puntos. De nuevo, la IA ha sido clave para entender el enunciado y asegurarse de que mi código cumplía lo pedido en el enunciado. En adición, me ha ayudado a depurar el código, detectar errores y posibles mejoras.